CODITY.AI TECH ASSIGNMENT SUBMISSION

Distributed Asynchronous Job Scheduler (Overdrive)

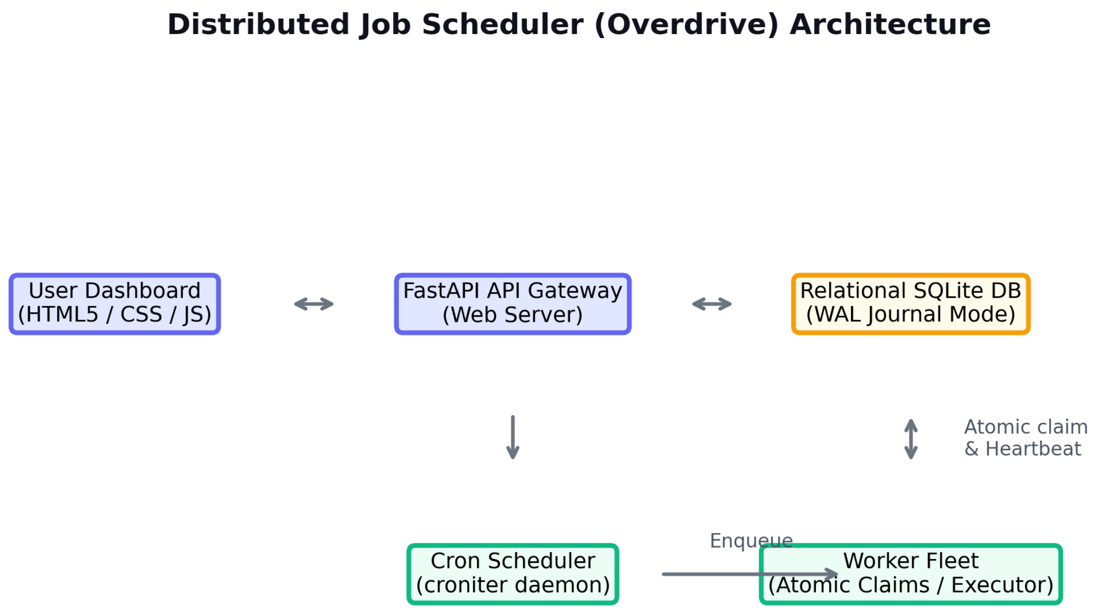
Full-Stack Design Thesis, Relational Schema Normalization, and Resiliency Verification.

SUBMISSION INFORMATION

Registration Number:	RA2311026010746
Candidate Name:	Prudhvi
Project Platform:	Distributed Job Scheduler (Overdrive)
GitHub Repository:	https://github.com/prudhvi98491/distributed-job-scheduler
Date of Submission:	July 5, 2026
Academic Institution:	SRM Institute of Science and Technology

1. System Architecture (20 Marks)

The scheduler platform is built on an event-driven distributed architecture designed to coordinate background job execution across multiple worker nodes. The system consists of three main blocks:
- The Web API Gateway (FastAPI): Handles user registration, queue updates, job enqueueing, and dashboard metrics.
- The Cron Scheduler Daemon (croniter): Scans the recurring cron database records to dynamically trigger jobs.
- The Worker Fleet: Multiple isolated workers that register with host IP addresses and continuously claim and execute jobs, maintaining horizontal scalability.



2. Database Design & Normalization (20 Marks)

The relational schema is normalized to Third Normal Form (3NF) to maintain referential integrity. Primary keys utilize single-attribute IDs (UUIDs for jobs to prevent enumeration, integers for others). Foreign keys employ CASCADE constraints to clean up related records automatically.

Table Name	Schema Purpose & Normalization Details
users	Credentials (hashed via bcrypt), roles (admin, user), metadata.
organizations	Enables multi-tenancy workspace isolation.
projects	Contains project workspaces owned by organizations.
queues	Defines concurrency, priorities, and retry policies.
jobs	Core queue table containing payload, status, and parent_job_id.
job_executions	Tracks execution details, duration (ms), and trace errors.
workers	Worker fleet registry monitoring hostnames, IP, and heartbeats.
cron_jobs	Schedules recurring cron jobs to dynamically queue jobs.
dead_letter_queue	Holds permanently failed tasks for manual replay.

3. Backend Engineering & API Design (25 Marks)

The REST APIs utilize Pydantic schemas for request validation. A JWT security layer manages authentication and role-based permissions (RBAC). The endpoints aggregate system-wide performance and queue throughput data.

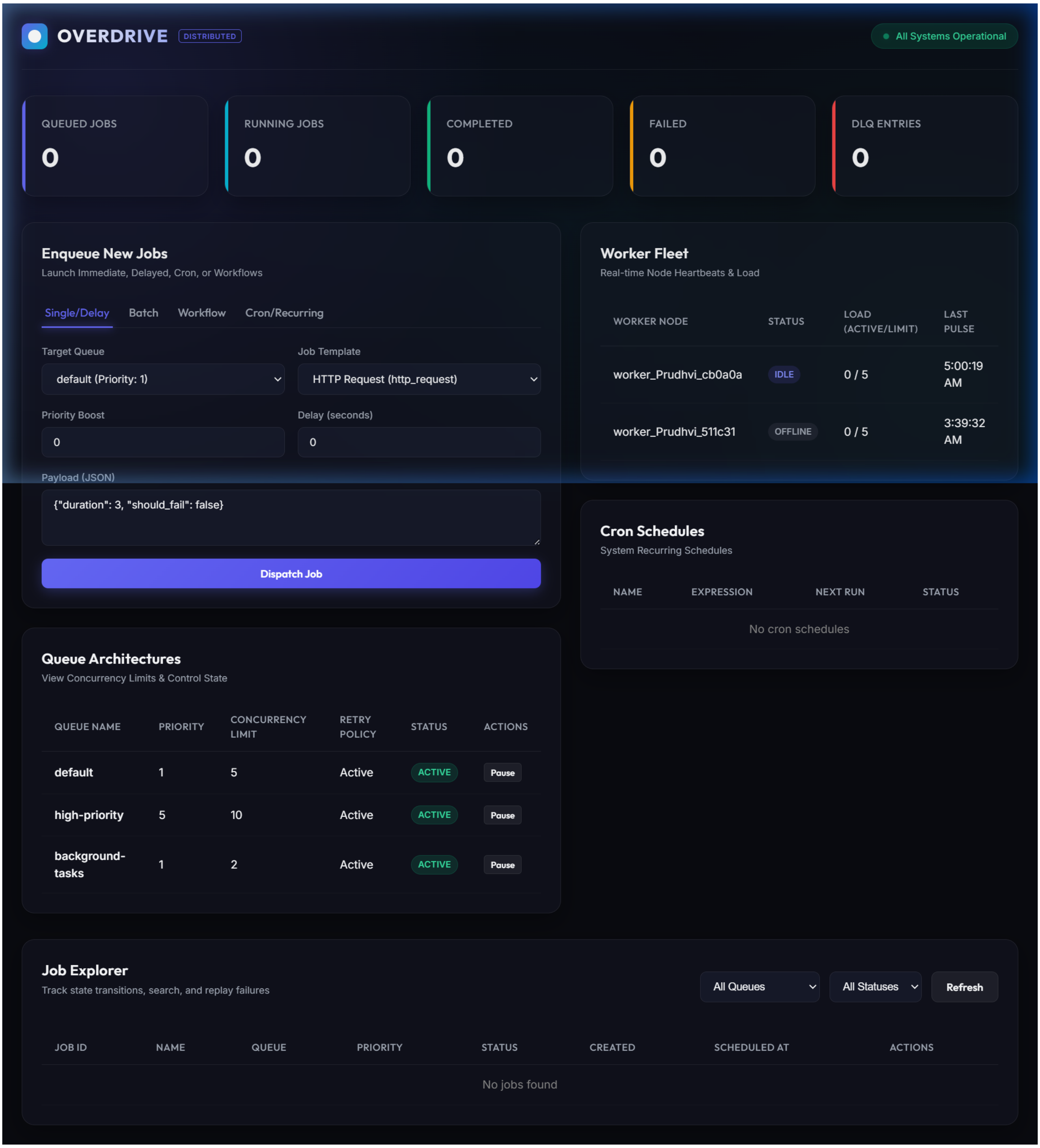
Method	**Endpoint Path**	**Description & Functionality**
POST	/api/auth/register	Registers a user account with hashed credentials.
POST	/api/auth/login	Authenticates credentials and issues a JWT token.
POST	/api/queues	Creates a queue specifying concurrency limit & priority.
PATCH	/api/queues/{id}	Updates queue configs or pauses/resumes queue state.
POST	/api/jobs	Enqueue single task (immediate, delayed, or workflow).
POST	/api/jobs/batch	Dispatches bulk job payloads under a single API call.
GET	/api/jobs	Lists jobs with pagination, queue, and status filters.
POST	/api/jobs/{id}/retry	Manually replays a failed or DLQ job.
GET	/api/metrics	Returns aggregate metrics, throughput trends, and error counts.

4. Reliability & Concurrency (15 Marks)

To guarantee atomic task claims and prevent duplicate job execution, we execute claim operations inside a SQLite transaction with an IMMEDIATE lock. Candidate jobs are filtered dynamically to only select from active queues whose current active job count is below their concurrency_limit.
Workers register themselves in the database and issue heartbeats every 5 seconds. If a worker goes offline for more than 15 seconds, it is marked as offline, and its active jobs are automatically re-queued for failover.

5. Frontend & UX (10 Marks)

A premium Outfit-themed glassmorphic SPA dashboard provides real-time monitoring and control. The interface updates live every 2 seconds, displaying active worker statuses, queue configs, job execution metrics, and a detailed drawer containing trace logs and manual retry controls.



6. Testing & Documentation (10 Marks)

We implemented automated integration tests in tests/test_scheduler.py to verify system correctness. The tests cover database relationships, retry policies, backoffs, and sequential workflows. Running pytest completes successfully:

```
tests/test_scheduler.py::test_database_seeding_and_relations PASSED
tests/test_scheduler.py::test_job_execution_lifecycle_and_retries PASSED
tests/test_scheduler.py::test_workflow_dependencies_unblocking PASSED
============================== 3 passed in 2.02s ==============================
```

Verification GitHub Repository Link

https://github.com/prudhvi98491/distributed-job-scheduler